Chapter 17 (Free-Space Management) — Homework (Simulation)

Zhiyuan Pan

All runs use `malloc.py` in `ostep-homework/vm-freespace/`. Questions 4–6 need statistics over long random runs, so I wrote `code/chapter17/malloc_stats.py`. It runs the official simulator with `-c` and only parses its output: failed allocations, elements searched, free-list length over time, and the largest free chunk. Everything can be reproduced with `code/chapter17/run.sh`.

How the simulator works (from reading `malloc.py`), with the defaults used below: a 100-byte heap at address 1000, random requests of 1–10 bytes, 50% of operations are allocations, and `-H 0` (no header).

- **Allocation**: the request is rounded up to the alignment (`-a`, off by default) and the header size (`-H`) is added. The policy picks a chunk: `BEST` = smallest chunk that fits, `WORST` = largest, `FIRST` = first one in list order. The chunk is split, and the remainder **stays in the same list position**.
- **"searched N elements"**: `BEST` and `WORST` always scan the whole list. `FIRST` stops at the first chunk that fits. A failed request (returned −1) scans the whole list for every policy.
- **Free**: the chunk is put back according to `-l` (`ADDRSORT`, `SIZESORT+`, `SIZESORT-`, `INSERT-FRONT`, `INSERT-BACK`). The list is only re-sorted on a free, so after a split a `SIZESORT` list can be briefly out of order.
- **Coalescing (`-C`)**: after each free, the simulator walks the list and merges an entry with the **next entry in the list** if their addresses are contiguous. So it only finds all neighbors when the list is sorted by address. This matters in Q5 and Q7.

Q1

> First run with the flags `-n 10 -H 0 -p BEST -s 0` to generate a few random allocations and frees. Can you predict what alloc()/free() will return? Can you guess the state of the free list after each request? What do you notice about the free list over time?

The free list starts as one chunk `[1000, size 100]`. Step by step (best fit, address-ordered list, no coalescing):

#	operation	returns	searched	free list afterwards	why
1	`ptr[0] = Alloc(3)`	1000	1	[1003:97]	split the only chunk
2	`Free(ptr[0])`	0	–	[1000:3] [1003:97]	no coalescing: the 3 bytes become a separate entry
3	`ptr[1] = Alloc(5)`	1003	2	[1000:3] [1008:92]	only the 97-byte chunk fits
4	`Free(ptr[1])`	0	–	[1000:3] [1003:5] [1008:92]	
5	`ptr[2] = Alloc(8)`	1008	3	[1000:3] [1003:5] [1016:84]	only the big chunk fits
6	`Free(ptr[2])`	0	–	[1000:3] [1003:5] [1008:8] [1016:84]	
7	`ptr[3] = Alloc(8)`	1008	4	[1000:3] [1003:5] [1016:84]	**exact fit**: the 8-byte chunk is used up and removed
8	`Free(ptr[3])`	0	–	[1000:3] [1003:5] [1008:8] [1016:84]	
9	`ptr[4] = Alloc(2)`	1000	4	[1002:1] [1003:5] [1008:8] [1016:84]	smallest chunk ≥ 2 is the 3-byte one; 1 byte left
10	`ptr[5] = Alloc(7)`	1008	4	[1002:1] [1003:5] [1015:1] [1016:84]	smallest chunk ≥ 7 is the 8-byte one; 1 byte left

Every value matched `-c`. What I notice about the free list over time:

- **It keeps growing and fragmenting.** Without coalescing, every free adds a new entry, even when the freed chunk is right next to free space (after step 2, `[1000:3]` and `[1003:97]` are adjacent but stay separate).
- **Best fit leaves tiny slivers.** Steps 9 and 10 each leave a 1-byte chunk that almost no future request can use. 91 bytes are free at the end, but in four pieces.
- **Best fit has to scan the whole list** on every allocation (4 elements by the end), even when the first chunk would have done.

Q2

> How are the results different when using a WORST fit policy to search the free list (`-p WORST`)? What changes?

Same seed and same requests (`-n 10 -H 0 -p WORST -s 0`). Steps 1–6 are identical, because only one chunk fits each request. The difference starts at step 7:

#	operation	BEST	WORST
7	`Alloc(8)`	1008 (exact 8-byte hole)	**1016** (carves the 84-byte chunk)
9	`Alloc(2)`	1000 (3-byte hole)	**1024** (largest chunk)
10	`Alloc(7)`	1008 (8-byte hole)	**1026** (largest chunk)

Final free lists:

```
BEST:   [1002:1] [1003:5] [1015:1] [1016:84]                    4 entries, 18 elements
searched in total
WORST:  [1000:3] [1003:5] [1008:8] [1016:8] [1033:67]           5 entries, 20 elements
searched in total
```

What changes:

- Worst fit **never reuses the small holes**; it keeps cutting pieces off the biggest chunk. The holes of 3, 5, 8 and 8 bytes just accumulate, so the list is longer (5 entries instead of 4) and the big chunk shrinks faster (67 instead of 84 bytes).
- It avoids the 1-byte slivers of best fit in this short run. But over a long run it is by far the worst policy (Q5): with 1000 operations and no coalescing, 78.6% of requests fail, and the heap ends up as about 100 separate 1-byte chunks.
- It still scans the whole list, so it searches as many elements as best fit (slightly more here, because its list is longer).

Q3

> What about when using FIRST fit (`-p FIRST`)? What speeds up when you use first fit?

With `-p FIRST`, every allocation happens to land in exactly the same place as with best fit, and the final free list is identical: `[1002:1] [1003:5] [1015:1] [1016:84]`. In this address-ordered list, the first chunk that fits is also the smallest one that fits.

What speeds up is the search:

operation	BEST searched	FIRST searched
`Alloc(3)`	1	1
`Alloc(5)`	2	2
`Alloc(8)`	3	3
`Alloc(8)`	4	**3** (stops at the 8-byte hole)
`Alloc(2)`	4	**1** (the first entry fits)
`Alloc(7)`	4	**3**
total	**18**	**13**

First fit stops at the first chunk that is big enough, instead of examining every free chunk to find the best (or worst) one. The cost is that it does not pick the tightest fit in general, and it tends to break up the chunks at the front of the list. Over 1000 operations (Q5) it fails more often than best fit (38.8% vs 24.9% without coalescing), but successful allocations look at far fewer elements (16.9 vs 31.6 on average).

Q4

> For the above questions, how the list is kept ordered can affect the time it takes to find a free location for some of the policies. Use the different free list orderings (`-l ADDRSORT`, `-l SIZESORT+`, `-l SIZESORT-`) to see how the policies and the list orderings interact.

Total elements searched for the 10-operation run from Q1 (`-n 10 -H 0 -s 0`):

policy	ADDRSORT	SIZESORT+	SIZESORT-
BEST	18	18	18
WORST	20	20	20
FIRST	13	13	**6**

Over a longer run (`-n 1000 -H 0`, no coalescing, mean of seeds 0–4):

policy	order	failed allocs	avg elements searched (all / successful)	avg free-list length
BEST	any of the three	24.9%	32.61 / 31.61	32.0
WORST	any of the three	78.6%	80.30 / 63.15	77.4
FIRST	ADDRSORT	38.8%	29.20 / 16.94	43.6
FIRST	SIZESORT+	**24.9%**	25.44 / 22.03	32.0
FIRST	SIZESORT-	77.8%	65.92 / **1.08**	76.6

How policy and ordering interact:

- **BEST and WORST don't care about the order.** They always scan the entire list, so the search cost is just the list length, and they pick the same chunk whatever the order. All three orderings gave identical results.
- **FIRST depends entirely on the order**, because the order decides which chunk is "first":
  - `SIZESORT+` (smallest first): the first chunk that fits is the smallest one that fits, so **first fit becomes best fit**: exactly the same failure rate (24.9%) and fragmentation as BEST, but it stops early (22.0 instead of 31.6 elements per successful search).
  - `SIZESORT-` (largest first): the first chunk is the largest, so **first fit becomes worst fit**, with the same terrible fragmentation (77.8% failures). A successful search almost always ends at the first element (1.08), but every failed request still has to scan the whole list.
  - `ADDRSORT`: a genuine first fit, between the two in both speed and fragmentation.

So a size-sorted list lets the cheap first-fit search reproduce the placement of best fit (or worst fit) without scanning the entire list. The price is keeping the list sorted on every free.

Q5

> Coalescing of a free list can be quite important. Increase the number of random allocations (say to `-n 1000`). What happens to larger allocation requests over time? Run with and without coalescing (i.e., without and with the `-C` flag). What differences in outcome do you see? How big is the free list over time in each case? Does the ordering of the list matter in this case?

`-n 1000 -H 0`, address-ordered list, mean of seeds 0–4:

policy	coalescing	failed allocs	avg free-list length	final largest free chunk
BEST	no	24.9%	32.0	6.4
BEST	`-C`	**4.9%**	**3.3**	59.6
WORST	no	78.6%	77.4	1.0
WORST	`-C`	**5.8%**	**4.2**	41.4
FIRST	no	38.8%	43.6	5.2
FIRST	`-C`	**5.3%**	**3.7**	51.6

Larger requests start failing over time without coalescing. Failure rate by request size, best fit, seed 0:

request size	1–6	7	8	9	10
no coalescing, first 25% of allocations	0–8%	6%	5%	50%	83%
no coalescing, last 25% of allocations	0%	18%	**100%**	**100%**	**100%**
`-C`, first 25%	0%	6%	0%	0%	0%
`-C`, last 25%	0%	0%	0%	0%	0%

Free-list length and largest free chunk over time (best fit, seed 0):

after operation	10	50	100	250	500	750	1000
list length, no coalescing	4	16	20	27	35	30	31
largest chunk, no coalescing	84	38	8	8	7	7	6
list length, `-C`	1	2	4	3	7	3	1
largest chunk, `-C`	91	81	29	35	14	32	98

Differences:

- **Without coalescing**, freed chunks are never merged back, so the heap is chopped into ever smaller pieces. After about 100 operations the largest free chunk is only 8 bytes, and from then on **every request of 8, 9 or 10 bytes fails**, even though 60–90 bytes are free in total. The free list grows to 30–40 entries (up to 100 with worst fit), which also makes each search slower.
- **With coalescing**, adjacent free chunks merge back into large ones. The list stays at 1–8 entries, large requests keep succeeding, and the few failures left happen when the heap is genuinely full. At the end of the seed-0 run the whole heap is back to almost one 98-byte chunk.

Yes, the ordering matters, because this simulator only merges chunks that are next to each other *in the list*. Same workload, with `-C`:

policy	ADDRSORT	SIZESORT+	SIZESORT-	INSERT-FRONT	INSERT-BACK
BEST, failed allocs	**4.9%**	21.4%	21.2%	19.4%	19.9%
BEST, avg list length	**3.3**	27.7	29.9	31.2	31.5
FIRST, failed allocs	**5.3%**	20.6%	72.5%	61.4%	44.6%

Only an address-sorted list puts neighboring chunks next to each other, so only `ADDRSORT` gets the full benefit of coalescing. With any other order, `-C` merges very little and the results are close to no coalescing at all. (A real allocator can coalesce with any list order by looking up the neighbors by address, e.g. with boundary tags, but this simulator does not.)

Q6

> What happens when you change the percent allocated fraction `-P` to higher than 50? What happens to allocations as it nears 100? What about as the percent nears 0?

`-n 1000 -H 0 -p BEST`, address-ordered, mean of seeds 0–4:

`-P`	allocs	failed (with `-C`)	avg list length (`-C`)	final free bytes (`-C`)	failed (no `-C`)	avg list length (no `-C`)
1	500	0.0%	1.0	100.0	0.0%	19.6
10	500	0.0%	1.0	96.0	0.0%	25.4
30	501	0.0%	1.2	95.0	5.8%	29.2
50	516	4.9%	3.3	63.6	24.9%	32.0
60	598	29.9%	4.6	21.4	35.9%	29.8
70	695	52.5%	4.6	13.0	54.1%	23.5
80	794	69.1%	3.5	6.2	69.5%	13.3
90	899	83.2%	1.5	2.0	83.0%	1.8
99	989	96.2%	0.1	0.2	96.2%	0.1
100	1000	98.0%	0.0	0.0	98.0%	0.0

- **Above 50%**, there are more allocations than frees, so the heap fills up. Failures rise quickly (30% at `-P 60`, 52% at 70, 83% at 90). The free list gets *shorter* rather than longer, simply because there is almost no free memory left. Now failures are caused by the heap being full, not by fragmentation, so coalescing hardly helps any more.
- **Near 100**, nothing is ever freed. The first ~20 requests use up the 100-byte heap (about 5.5 bytes each), and then every request fails: 98% at `-P 100`. The free list becomes empty, so a failed search examines 0 elements.

- **Near 0**, almost every operation wants to be a free, but a free is only possible if something is allocated. The workload therefore degenerates into strict alternation: allocate, free, allocate, free (exactly 500 allocations in 1000 operations). At most one block is in use at a time, so nothing ever fails. With coalescing the free list stays a single chunk. Without coalescing, though, every free still adds a new entry, so the list still grows to about 20 entries.

Q7

> What kind of specific requests can you make to generate a highly-fragmented free space? Use the `-A` flag to create fragmented free lists, and see how different policies and options change the organization of the free list.

1. Free every other block. Fill the heap with ten 10-byte blocks, then free blocks 0, 2, 4, 6, 8, and ask for 20 bytes:

```
./malloc.py -H 0 -C -A +10,+10,+10,+10,+10,+10,+10,+10,+10,+10,-0,-2,-4,-6,-8,+20 -c
ptr[10] = Alloc(20) returned -1 (searched 5 elements)
Free List [ Size 5 ]: [ addr:1000 sz:10 ][ addr:1020 sz:10 ][ addr:1040 sz:10 ][ addr:1
060 sz:10 ][ addr:1080 sz:10 ]
```

Half the heap (50 bytes) is free, yet a 20-byte request fails. This is **external fragmentation**, and coalescing (`-C` is on) cannot help, because no two free chunks are neighbors. Freeing the block in between (`-1`) changes that: blocks 0, 1 and 2 merge into one 30-byte chunk `[1000:30]`, and `Alloc(20)` succeeds at 1000.

2. Leave holes of different sizes, then compare policies. Allocate 20, 5, 15, 5, 30, 5, 20 bytes (exactly 100), free the 20-, 15- and 30-byte blocks (holes of 20, 15 and 30 bytes, 65 bytes free), then request 12, 18 and 28 bytes:

```
./malloc.py -H 0 -p POLICY -A +20,+5,+15,+5,+30,+5,+20,-0,-2,-4,+12,+18,+28 -c
```

policy	Alloc(12)	Alloc(18)	Alloc(28)	final free list
BEST	15-byte hole (3 left)	20-byte hole (2 left)	30-byte hole (2 left)	[1018:2] [1037:3] [1073:2]
WORST	30-byte hole (18 left)	20-byte hole (2 left)	**fails**	[1018:2] [1025:15] [1057:18]
FIRST	20-byte hole (8 left)	30-byte hole (12 left)	**fails**	[1012:8] [1025:15] [1063:12]

Best fit packs all three requests and leaves only 7 bytes, split into 2-, 3- and 2-byte slivers. Worst fit and first fit each "waste" the large hole on a small request, so the 28-byte request no longer fits anywhere, even though 35 bytes are still free.

3. List ordering vs coalescing. Allocate 10, 20, 30, 40 bytes and free the first three (they are contiguous: 1000–1059), with `-C`:

`-l`	**final free list**
ADDRSORT	[1000:60] — fully merged
SIZESORT-	[1030:30] [1010:20] [1000:10] — **not merged**

In size order, contiguous chunks are not neighbors in the list, so the simulator's coalescing misses them and leaves three pieces instead of one 60-byte chunk.

Summary. Fragmentation comes from (a) interleaving allocations and frees so that live blocks separate the free holes (no coalescing can fix that), (b) a placement policy that wastes large holes (WORST, and often FIRST) or leaves slivers (BEST), and (c) not coalescing, or coalescing with a list that is not address-ordered. A random workload can do this too: worst fit with no coalescing (`-n 1000 -p WORST`) ends with about 100 one-byte chunks.

Simulator note: in `-A` mode, freeing the index of an allocation that failed crashes `malloc.py` (the index is never recorded), so the sequences above only free blocks that were allocated successfully.